Lecture 22 Practice Questions and Solutions

Question 1 (q1_sol.py)

Write a program that repeatedly asks the user to enter a term *code number* and the prints the corresponding semester name. It should stop when the user types <return>, and print a helpful message if an invalid code is typed. Use a dictionary to efficiently store the semester codes.

Here are the semester codes:

```
Semester   Code
---------------
 Spring      1
 Summer      4
 Fall        7
```

Sample Run

```
Enter semester code: 4
4 is Summer

Enter semester code: 8
8 is invalid

Enter semester code: 1
1 is Spring

Enter semester code: <return>
All done!
```

Sample Solution

```
codes = {1: 'Spring', 4: 'Summer', 7: 'Fall'}

while True:
    code = input('Enter semester code: ')
    code = code.strip() # remove any whitespace from the input
    if code == '':
      break
    code = int(code) # convert the input to an integer
    if code in codes:
        semester = codes[code]
        print(f'{code} is {semester}')
    else:
        print(f'{code} is invalid')
    print()

print('All done!')
```

Question 2 (q2_sol.py)

SFU term codes are 4-digit numbers, where each number has a specific meaning. For example, the term code 1267 means Fall 2026 because:

- the first digit is a 1 (for this century)
- the next two digits are 26, the last two digits of the year 2026
- the last digit is a 7, which is the semester code for Fall

The semester codes are:

```
Semester   Code
---------------
 Spring      1
 Summer      4
 Fall        7
```

Write a program that reads in a term code and prints the year and term as shown.

Sample Runs

```
Term code: 1267
Fall 2026

Term code: 1161
Spring 2016

Term code: 1274
Summer 2027

Term code: <enter>
done!
```

Sample Solution

```
codes = {1: 'Spring', 4: 'Summer', 7: 'Fall'}

while True:
    # get the term code from the user
    term_code = input("Enter term code: ")
    term_code = term_code.strip()

    if term_code == '':
        break

    # get the year from the term code
    year = "20" + term_code[1] + term_code[2]
```

```
    # get the term from the term code
    term_num = int(term_code[3])
    term = codes[term_num]

    # print the year and term
    print(f"{term} {year}")
    print()

print("done!")
```

Question 3 (q3_sol.py)

Write a program that asks the user for the following information:

- the name of a file they would like to create
- a list of names they would like to appear in the file, e.g. the user might enter the five names "Ava Ben Cara Dana Evan"
- ask for how many names they want in the file

After getting this information, the program should create a file with the given name and then fill it with random names (from the user's list) until the file contains the specified number of names. Each name should be on its own line.

Hint: import the random module, and use random.choice() to select random names from the list.

Sample Run

```
What file? votes.txt
What names? Ava Ben Cara Dana Evan
How many names in the file? 100

Okay, created votes.txt with 100 random names.
```

Now the file votes.txt should exist and contain 100 random names, e.g. starting like this:

```
Dana
Dana
Ava
Cara
Evan
Ava
Evan
Ben
Ava
Ava
...
```

There should be exactly 100 names in random order.

Sample Solution

```
import random

#
# get the information from the user
#
file_name = input("What file? ")
names = input("What names? ").split() # split() chops a string into a list
```

```
num_names = int(input("How many names in the file? "))

#
# open the file and write the random names to it
#
name_file = open(file_name, 'w') # 'w' opens the file for writing
for i in range(num_names):
    rand_name = random.choice(names) # choose a random name from the list
    name_file.write(rand_name + '\n') # write the name, then a newline

#
# close file and print a message to the user
#
name_file.close()
print(f"Okay, created {file_name} with {num_names} random names.")
```

Question 4 (q4_sol.py)

The student council president election has five candidates: Ava, Ben, Cara, Dana, and Evan. The file votes.txt contains all the votes cast, one per line. Any name other than one of these five candidates is invalid and counts as a spoiled vote.

Write a program that reads the votes from votes.txt and counts how many votes each candidate received, and then prints out the results from most votes to least votes. Also include the number of spoiled votes. If there is a tie, then order doesn't matter.

For example, suppose votes.txt contains this:

```
Cara
Dana
Ava
Dana
Ben
Carl
Ben
Ben
Ava
Scooby
Ben
```

Then the program should print:

```
Ben: 4
Dana: 2
Ava: 2
Cara: 1
Evan: 0
Spoiled votes: 2
```

Sample Solution

```
#
# initialize all the counts to 0
#
votes = {'Ava': 0, 'Ben': 0, 'Cara': 0, 'Dana': 0, 'Evan': 0}
spoiled_votes = 0

#
# open the file and read the names (one per line)
#
print("Reading votes from votes.txt...")
vote_file = open("votes.txt", 'r') # open the file for reading
for name in vote_file:
    name = name.strip() # remove whitespace from the name
    if name in votes:
```

```
        votes[name] += 1 # increment the count for this candidate
    else:
        spoiled_votes += 1 # increment the count for spoiled votes

#
# make list of pairs of [count, name] so we can sort them
#
name_vote = []
for name in votes:
    pair = [votes[name], name] # create a pair of [count, name]
    name_vote.append(pair)     # add it to the end of the list of all pairs

# .sort() sorts the list of pairs from least votes to most votes
name_vote.sort()
print('name_vote:', name_vote)

# reverse the order so they go from most votes to least votes
name_vote.reverse()

#
# print the results
#
for count, name in name_vote:
    print(f"{name}: {count}")
print(f"Spoiled votes: {spoiled_votes}")
```